

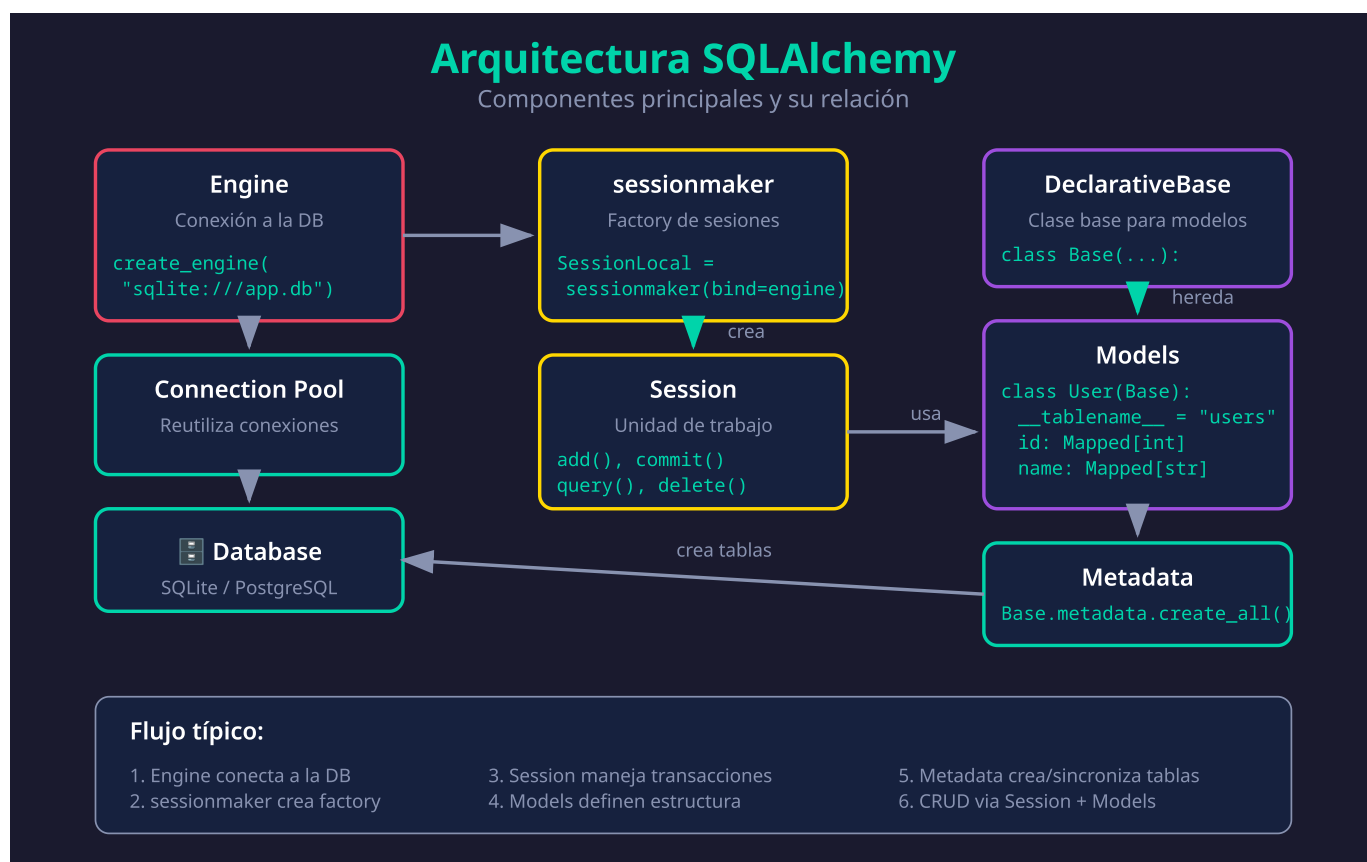
⚙️ Configuración de SQLAlchemy

🎯 Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Crear un Engine de conexión a base de datos
- ☒ Configurar sesiones para operaciones
- ☒ Definir la Base declarativa para modelos
- ☒ Entender el flujo de conexión de SQLAlchemy

📖 Contenido



1. Componentes Principales

SQLAlchemy tiene tres componentes clave:

Componente	Propósito	Analogía
Engine	Conexión a la base de datos	"La fábrica de conexiones"
Session	Unidad de trabajo (transacciones)	"El carrito de compras"
Base	Clase padre de todos los modelos	"El molde de galletas"

2. El Engine

El **Engine** es el punto de entrada a la base de datos. Gestiona el pool de conexiones.

```

from sqlalchemy import create_engine

# SQLite (archivo local)
engine = create_engine(
    "sqlite:///./app.db",
    echo=True # Muestra SQL generado (útil para debug)
)

# SQLite en memoria (para tests)
engine = create_engine("sqlite:///memory:")

# PostgreSQL (producción)
engine = create_engine(
    "postgresql://user:password@localhost:5432/dbname",
    pool_size=5, # Conexiones en el pool
    max_overflow=10, # Conexiones extra permitidas
    pool_pre_ping=True # Verifica conexiones antes de usar
)

```

URLs de Conexión

```

# Formato general
# dialect+driver://username:password@host:port/database

# SQLite
"sqlite:///./app.db" # Archivo relativo
"sqlite:///absolute/path.db" # Archivo absoluto
"sqlite:///memory:" # En memoria

# PostgreSQL
"postgresql://user:pass@localhost:5432/mydb"
"postgresql+psycopg://user:pass@localhost:5432/mydb" # Con driver específico

# MySQL
"mysql+pymysql://user:pass@localhost:3306/mydb"

```

3. La Session

La **Session** es donde ocurre la magia. Gestiona las transacciones y el estado de los objetos.

```

from sqlalchemy.orm import Session, sessionmaker

# Opción 1: Crear session directamente
with Session(engine) as session:
    # Operaciones aquí
    session.commit()

# Opción 2: Crear una fábrica de sessions (recomendado)
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False, # Control manual de transacciones
    autoflush=False # Control manual de flush
)

```

```
# Usar la fábrica
with SessionLocal() as session:
    # Operaciones aquí
    session.commit()
```

Ciclo de Vida de una Session

```
# 1. Crear session
session = SessionLocal()

try:
    # 2. Realizar operaciones
    user = User(name="John")
    session.add(user)

    # 3. Commit - persiste los cambios
    session.commit()

except Exception:
    # 4. Rollback en caso de error
    session.rollback()
    raise

finally:
    # 5. Cerrar session
    session.close()

# Forma pythónica con context manager
with SessionLocal() as session:
    user = User(name="John")
    session.add(user)
    session.commit() # Rollback automático si hay excepción
```

4. La Base Declarativa

La **Base** es la clase de la que heredan todos tus modelos:

```
from sqlalchemy.orm import DeclarativeBase

# SQLAlchemy 2.0 - Estilo moderno
class Base(DeclarativeBase):
    pass

# Todos los modelos heredan de Base
class User(Base):
    __tablename__ = "users"
    # ...

class Product(Base):
    __tablename__ = "products"
    # ...
```

Crear las Tablas

```
# Crea todas las tablas definidas en modelos que heredan de Base
Base.metadata.create_all(bind=engine)

# También puedes eliminarlas (¡cuidado en producción!)
Base.metadata.drop_all(bind=engine)
```

5. Estructura de Archivos Recomendada

```
src/
├─ database.py      # Configuración de SQLAlchemy
├─ models.py        # Modelos (tablas)
├─ schemas.py       # Schemas Pydantic
└─ main.py          # FastAPI app
```

database.py

```
"""
Database configuration for SQLAlchemy
"""
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

# URL de conexión (usar variables de entorno en producción)
SQLALCHEMY_DATABASE_URL = "sqlite:///./app.db"

# Crear engine
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # Solo para SQLite
)

# Fábrica de sessions
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False
)

# Base declarativa
class Base(DeclarativeBase):
    pass

def get_db():
    """
    Dependency que provee una session de base de datos.
    Se usa en FastAPI con Depends().
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

6. Configuración con Variables de Entorno

```
# database.py - versión con configuración externa
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

# Leer de variable de entorno con fallback
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "sqlite:///./app.db" # Default para desarrollo
)

# Ajustes específicos por base de datos
connect_args = {}
if DATABASE_URL.startswith("sqlite"):
    connect_args["check_same_thread"] = False

engine = create_engine(
    DATABASE_URL,
    connect_args=connect_args,
    echo=os.getenv("SQL_ECHO", "false").lower() == "true"
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass
```

.env

```
# Desarrollo
DATABASE_URL=sqlite:///./app.db
SQL_ECHO=true

# Producción
DATABASE_URL=postgresql://user:pass@db:5432/myapp
SQL_ECHO=false
```

7. Verificar la Conexión

```
# test_connection.py
from sqlalchemy import text
from database import engine

def test_connection():
    """Verifica que la conexión funciona"""
    with engine.connect() as connection:
        result = connection.execute(text("SELECT 1"))
        print(f"Conexión exitosa: {result.scalar()}")
```

```
if __name__ == "__main__":
    test_connection()
```

Ejemplo Completo

```
# database.py
from sqlalchemy import create_engine, text
from sqlalchemy.orm import DeclarativeBase, sessionmaker

SQLALCHEMY_DATABASE_URL = "sqlite:///./example.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    echo=True # Ver SQL en consola
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Verificar conexión
if __name__ == "__main__":
    with engine.connect() as conn:
        result = conn.execute(text("SELECT sqlite_version()"))
        print(f"SQLite version: {result.scalar()}")
    print("✅ Conexión configurada correctamente")
```

Checklist

- ☐ Entiendo qué es el Engine
- ☐ Sé crear y usar Sessions
- ☐ Puedo definir la Base declarativa
- ☐ Conozco la estructura de archivos recomendada

[← Anterior: Introducción a ORM](#) | [Siguiendo: Modelos Declarativos →](#)